Software Engineering Lab

Lab 16

Dependency Injection

Pisahkan Konstruksi,
Uji Tanpa Beban



lukman-ss

Masalah: Hard-Coded Infrastructure di Logika Bisnis

Instansiasi dependensi langsung di dalam domain mengunci kode pada provider konkret.

MOTIVASI

Ketika Processor memanggil new RealGateway() di dalamnya, logika bisnis terkunci pada implementasi teknis eksternal. Perubahan vendor atau protokol jaringan memaksa rewrite pada lapisan domain.

BATAS LAB

Lab membuktikan manual wiring di Go standar dan isolasi unit test dengan mock. Tidak menggunakan framework DI otomatis (reflection/code-gen) atau manajemen lifecycle container rumit.

Separation of Configuration from Use is more critical than the choice between DI and Service Locator.

Mengapa Ini Krusial: Biaya Coupling Jangka Panjang

Keterikatan keras merusak kemampuan evolusi sistem dan melipatgandakan biaya pengujian.

DAMPAK PENGUJIAN

Setiap unit test terpaksa membuka koneksi jaringan, bergantung pada third-party gateway, berjalan lambat, dan menjadi flaky saat jaringan tidak stabil.

DAMPAK ARSITEKTUR

Domain logic kehilangan portabilitas. Mengganti vendor gateway memaksa modifikasi di banyak tempat karena tidak ada abstraksi antarmuka yang melindungi inti bisnis.

Tight coupling turns simple unit tests into brittle, slow integration tests.

Mental Model: Externalize Object Creation

Pisahkan tanggung jawab membuat objek dari tanggung jawab menjalankan logika bisnis.

PRINSIP UTAMA

Komponen domain tidak boleh memutuskan bagaimana dependensinya diciptakan. Siapa yang membutuhkan dependensi cukup mendeklarasikannya; pihak luar (assembler) yang menyediakannya.

ANALOGI PERANGKAT KERAS

Seperti stopkontak listrik standar: peralatan elektronik tidak perlu tahu apakah listrik berasal dari panel surya atau generator PLN. Cukup colokkan kabel ke soket yang sesuai.

Don't let business logic manufacture its own dependencies.

Inversion of Control vs Dependency Injection

Memahami payung konsep arsitektur besar dan teknik spesifik yang menerapkannya.

INVERSION OF CONTROL (IoC)

Prinsip desain umum di mana kendali alur eksekusi atau instansiasi diserahkan ke luar objek (Hollywood Principle: "Don't call us, we'll call you").

DEPENDENCY INJECTION (DI)

Pola implementasi konkret dari IoC di mana dependensi diberikan (di-inject) ke dalam objek dari luar melalui constructor, setter, atau interface.

IoC is the architectural philosophy; DI is the concrete delivery mechanism.

Kontrak Abstraksi: Interface PaymentGateway

Interface didefinisikan oleh consumer domain untuk membatasi ketergantungan.

KONTRAK KODE

```
type PaymentGateway interface {
    Charge(amount Money) error
}
```

PRINSIP GO IDIOMATIK

Konsumen (Processor) mendefinisikan interface yang ia butuhkan, bukan provider. Ini menjaga interface tetap ramping dan mudah di-mock secara presisi tanpa polusi metode.

Accept interfaces, return structs. Let the consumer dictate the boundary.

Constructor Injection: Eksplisit dan Aman

Menerima dependensi wajib saat instansiasi memastikan state objek selalu valid.

KODE CONSTRUCTOR

```
func NewProcessor(g PaymentGateway) *Processor {
    return &Processor{gateway: g}
}
```

MANFAAT KOMPILASI

Compiler Go memastikan dependensi tidak boleh tertinggal. Objek Processor dijamin tidak akan mengalami runtime nil pointer exception saat memanggil gateway.

Explicit constructor parameters make missing dependencies impossible at compile time.

Service Locator Anti-Pattern di Go

Menyembunyikan dependensi di balik container generik menciptakan ilusi fleksibilitas.

ANTI-PATTERN

```
func NewBadProcessor(c Container) *BadProcessor {
    return &BadProcessor{container: c}
}
```

MASALAH TERSEMBUNYI

Dependensi nyata tersembunyi di dalam runtime query method container. Compiler tidak bisa memeriksa apakah instance PaymentGateway benar-benar terdaftar di container.

Service Locators hide dependencies behind runtime queries and break compile-time safety.

Komparasi Arsitektur: DI vs Service Locator

Perbandingan langsung karakteristik teknis antara kedua pendekatan instansiasi.

CONSTRUCTOR INJECTION

- Visibilitas: Terlihat jelas di signature fungsi
- Keamanan: Compile-time type safe
- Testing: Mock dipassing langsung tanpa container setup

SERVICE LOCATOR

- Visibilitas: Tersembunyi di dalam method body
- Keamanan: Runtime panic jika key tidak ada
- Testing: Wajib register mock ke dalam container

Explicitness in architecture eliminates runtime surprises in production.

Peran Assembler / Composition Root

Titik sentral aplikasi di mana semua dependensi konkret dirakit sebelum digunakan.

MAIN.GO WIRING

```
func main() {
    gw := &RealGateway{}
    p := NewProcessor(gw)
    p.Process(Money{Amount: 100})
}
```

ISOLASI WIRING

Hanya fungsi main() yang mengetahui tipe konkret RealGateway. Semua kode domain hanya mengenal abstraksi interface PaymentGateway.

Confine knowledge of concrete infrastructure to the application's composition root.

Validasi Input: Guard Sebelum Panggilan Eksternal

Memastikan data domain valid sebelum menguras sumber daya infrastruktur.

GUARD CLAUSE

```
if amount.Amount ⩽ 0 {
    return ErrInvalidAmount
}
```

EFISIENSI SUMBER DAYA

Transaksi invalid ditolak seketika di lapisan memori tanpa perlu membuang koneksi jaringan ke payment gateway pihak ketiga.

Validate domain rules early to protect downstream infrastructure from wasteful calls.

Error Propagation yang Transparan

Menjaga error dari infrastructure gateway tetap transparan dan dapat ditangani caller.

PENANGANAN ERROR

```
if err := p.gateway.Charge(amount); err ≠ nil {
    return fmt.Errorf("charge failed: %w", err)
}
```

OBSERVABILITAS

Error wrapping menggunakan %w mempertahankan error sentry asli sambil memberikan konteks kontekstual bagi observability tracing sistem.

Wrap infrastructure errors cleanly without swallowing root diagnostic causes.

Mocking Terisolasi Tanpa Jaringan

Bagaimana interface memungkinkan pengetesan instan dan deterministik tanpa dependency luar.

STRUCT MOCK

```
type MockGateway struct {
    ChargeFunc func(Money) error
}
```

FLEKSIBILITAS PENGUJIAN

Setiap skenario pengujian (sukses, kegagalan vendor, timeout) dapat disimulasikan secara presisi dalam hitungan mikrodetik tanpa server nyata.

Interfaces enable instant, deterministic unit tests without network friction.

Cakupan Test Suites: 6 Kasus Terverifikasi

Rincian 6 test cases yang membuktikan ketangguhan Processor dan BadProcessor.

PENGUJIAN POSITIF & NEGATIF

- Process Success: Injeksi mock sukses (200 OK)
- Invalid Amount: Penolakan nilai negatif atau nol
- Gateway Failure: Simulasi error upstream gateway

SERVICE LOCATOR & VALUE OBJECTS

- BadProcessor Success: Resolusi via container
- BadProcessor Missing: Error saat key kosong
- Money Immutability: Value object valid & aman

Comprehensive test assertions verify both golden paths and failure handling.

`tests/processor_test.go:28-95`

Verifikasi Runtime: Race Detection Bersih

Menjalankan eksekusi tes dengan race detector aktif membuktikan keamanan konkurensi.

EKSEKUSI PERINTAH

```
go test -race ./...
PASS: TestProcessor (0.00s)
PASS: TestBadProcessor (0.00s)
ok  labs/16-dependency-injection/tests
```

JAMINAN THREAD-SAFETY

Instansiasi objek yang bersih dan stateless dependency injection tidak meninggalkan data race di lingkungan concurrent Go.

Zero shared mutable state in injected services guarantees race-clean execution.

Batas DI: Value Objects vs Service Objects

Kapan sebuah struct perlu di-inject dan kapan cukup diinstansiasi langsung.

SERVICE OBJECTS (BUTUH DI)

Objek dengan status stateful, koneksi IO, atau dependensi eksternal (Database, Gateway, Logger, Cache). Dikelola melalui interface injection.

VALUE OBJECTS (BYPASS DI)

Objek pembawa data murni tanpa behavior IO (struct Money, koordinat, ID). Instansiasi langsung tanpa melalui container DI.

Inject infrastructure and services; instantiate pure value objects directly.

Lifecycle Dependensi: Singleton vs Transient

Memahami siklus hidup objek yang diinjeksi ke dalam graph aplikasi.

SINGLETON

Satu instance dibagi bersama untuk seluruh runtime aplikasi (contoh: DB connection pool, HTTP client). Menghemat alokasi memori dan koneksi socket.

TRANSIENT / SCOPED

Instance baru dibuat per request atau per pemanggilan (contoh: transaction context, request tracer). Mencegah kebocoran data antar thread.

Match dependency lifecycle to its concurrency model and resource cost.

Pitfalls: Circular Dependency & Interface Pollution

Jebakan arsitektur umum saat merancang kode berbasis Dependency Injection.

CIRCULAR DEPENDENCY

Paket A mengimpor Paket B, Paket B membutuhkan Paket A. Go secara tegas menolak kompilasi package import cycle pada build time.

INTERFACE POLLUTION

Membuat interface untuk setiap struct tanpa kebutuhan mocking nyata membuat navigasi kode berbelit dan menambah beban kognitif tim.

Don't introduce an interface until you have at least two implementations or a test boundary.

Manual Wiring vs Automated DI Frameworks

Kapan manual wiring di main.go cukup dan kapan framework dibutuhkan.

MANUAL WIRING (LAB 16)

Kode sederhana, explicit, mudah di-trace via Go jump-to-definition, tidak ada magic reflection, waktu kompilasi instan.

FRAMEWORK DI (WIRE / DIG / FX)

Berguna jika graph dependensi memiliki ratusan service dengan lifecycle kompleks. Code generation (Wire) lebih disukai daripada reflection di Go.

Start with explicit manual wiring; adopt frameworks only when graph size demands it.

Batas Bukti Implementasi Lab 16

Kejujuran teknis mengenai batasan ruang lingkup yang dieksplorasi di lab ini.

APA YANG DIBUKTIKAN

Kontras Constructor Injection vs Service Locator, isolasi unit testing dengan mock, proteksi validasi input domain.

APA YANG DI LUAR CAKUPAN

Lab ini tidak menguji reflection framework, auto-wiring runtime, atau dynamic plugin loader skala enterprise.

A clear technical lab is honest about its explicit testing boundaries.

Checklist Produksi: Menerapkan DI yang Sehat

Panduan ringkas untuk tim engineering sebelum merilis kode ke produksi.

ATURAN DESAIN

1. Apakah constructor menerima interface, bukan struct konkret?
2. Apakah objek menolak pembuatan saat dependensi nil?
3. Apakah composition root terpusat di main?

ATURAN PENGUJIAN

4. Apakah setiap unit test dapat berjalan tanpa jaringan?
5. Apakah mock meng-assert error path secara presisi?
6. Apakah value object bebas dari injeksi container?

Healthy DI balances explicit construction, fast test isolation, and architectural clarity.

Ringkasan Arsitektur: Pilar 1 & 2

Dua pilar pertama yang memandu implementasi Dependency Injection berkualitas tinggi.

1. DECOUPLING

Pisahkan konstruksi dari penggunaan untuk melindungi aturan bisnis dari gejolak perubahan infrastruktur dan vendor eksternal.

2. TESTABILITY

Gunakan interface minimal pada consumer boundary agar test suite berjalan instan, deterministik, dan zero-network.

Externalize object creation. Preserve domain logic purity above all infrastructure concerns.

Ringkasan Arsitektur: Pilar 3 & 4

Dua pilar pelengkap untuk menjaga arsitektur tetap bersih dan mudah dipelihara.

3. EXPLICITNESS

Constructor injection membuat semua dependency terlihat jelas sejak kompilasi. Hindari Service Locator yang menyembunyikan query runtime.

4. SIMPLICITY

Prioritaskan manual wiring sederhana di composition root (main.go). Hindari framework automasi berlebihan jika graph objek masih mudah dilacak.

Prefer clear, explicit wiring over clever automations that obscure the object graph.

Eksternalisasi Pembuatan, Lindungi Logika Domain.

Externalize object creation. Keep domain logic pure and tests zero-network.

> Software Engineering Lab 16 — Dependency Injection
>
> `internal/di/gateway.go` • `internal/di/processor.go` • `internal/di/locator.go`
> `tests/processor_test.go` • `cmd/demo/main.go` • `README.md`
>
> REPOSITORY
> `github.com/lukman-ss/software-engineering-lab/tree/main/labs/16-dependency-injection`
>
> **Lukman / lukman-ss**
>
> Sumber: commit 8afe2a824667528d16d6d9b2ac434a18a6ac2db1 • Verified Go 1.22 (Race-Clean)